Warming Strategies and Database Protection

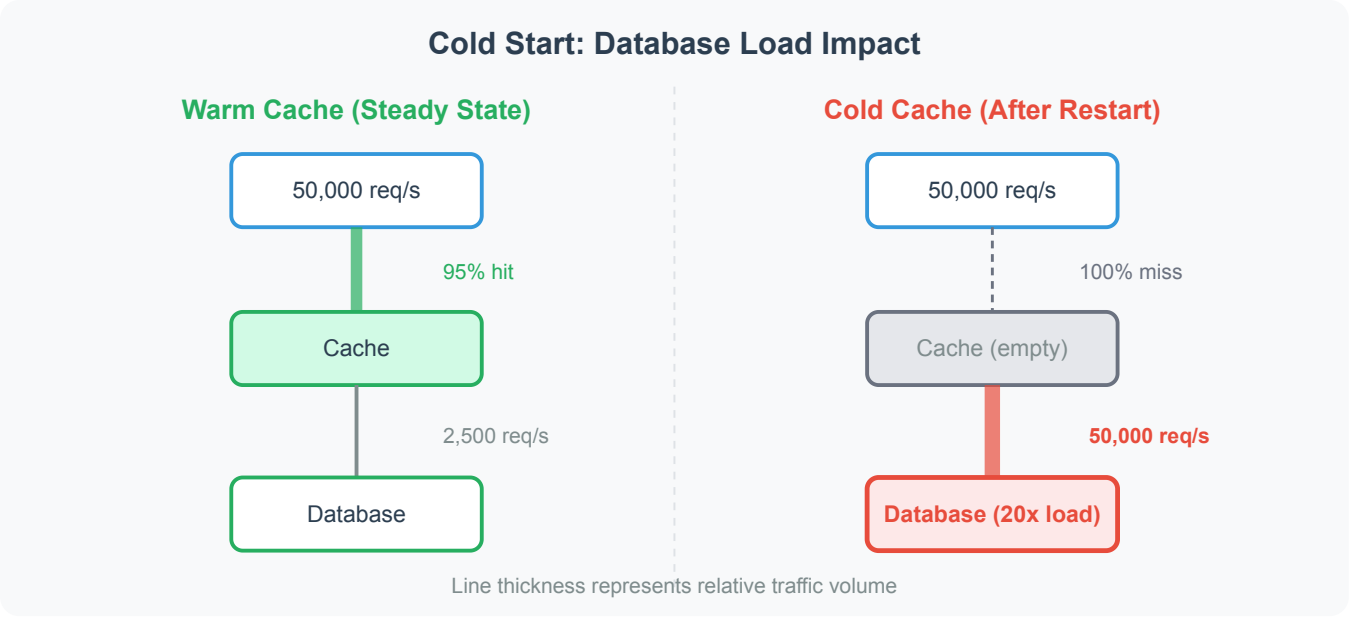
systemdesignschool.io/fundamentals/cache-cold-start

Cold Start

Replication and failover keep the cache serving during node failures. But when a replacement node comes online—or a new node joins the cluster—its memory is empty. Every request to that node misses. The database absorbs load it hasn't seen since the cache was first deployed.

The Problem

A system handling 50,000 reads per second with a 95% hit rate sends roughly 2,500 queries per second to the database. The database is provisioned for this load. When the cache is cold, all 50,000 requests become misses. The database sees a 20x spike instantly.



This isn't just a performance issue—it can cascade. The database saturates, query latency spikes, application threads block waiting for responses, connection pools exhaust, and the system returns errors to users. The cache was protecting the database. Removing that protection, even temporarily, can trigger an outage.

When Cold Starts Happen

Fresh deployments are the obvious case, but cold starts occur in several scenarios:

Node replacement after failure. The HA system promotes a replica or spins up a fresh node. If the replacement has no data (fresh instance rather than promoted replica), it starts cold.

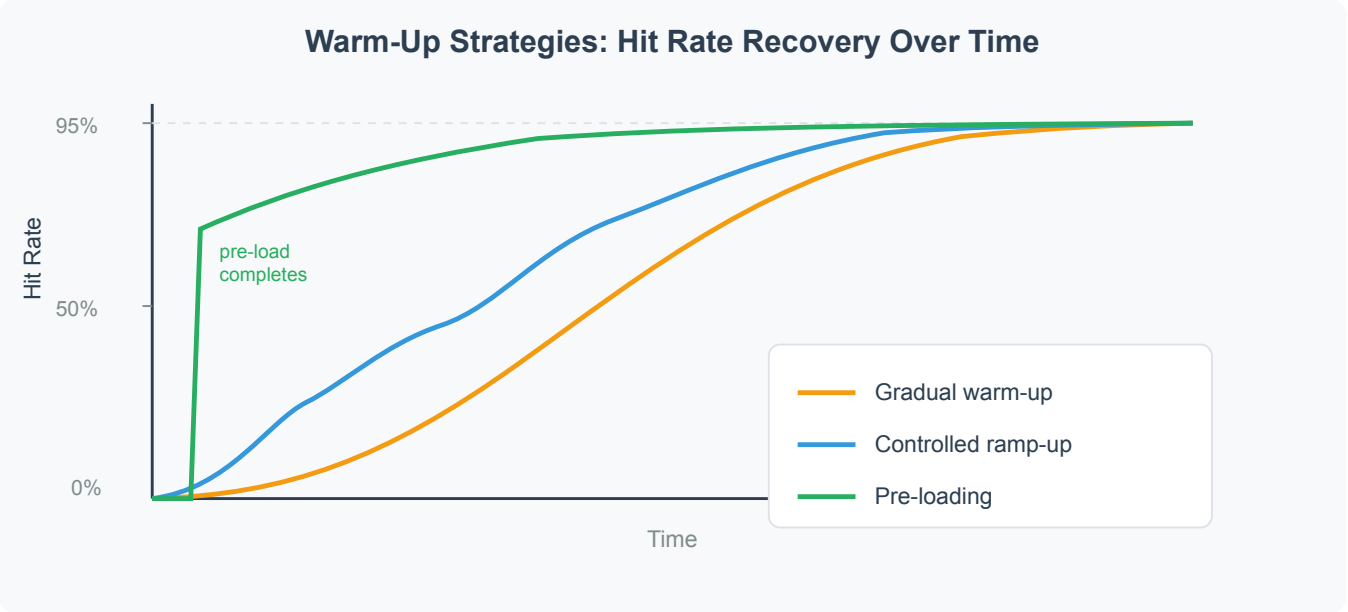
Scaling out. Adding a new shard to a Redis Cluster redistributes hash slots. Keys migrated to the new shard must be fetched from the database on first access—the new node doesn't have them in memory yet.

Cache flush after a bug. A code bug writes corrupt data to cache. The fix involves flushing all affected keys (or the entire cache). The system returns to cold-start conditions.

Mass TTL expiration. If many keys share the same TTL and were written at the same time (a batch import, for example), they expire simultaneously. The cache effectively goes cold for that key range. Adding jitter to TTLs (covered in the invalidation article) prevents this.

Warm-Up Strategies

Three approaches, from simplest to most proactive:



Gradual warm-up. Let natural traffic populate the cache through misses. The first request for each key hits the database; subsequent requests hit cache. The hit rate climbs from 0% toward steady-state over minutes as popular keys accumulate.

This works when the database has headroom to absorb the temporary spike. For the example above, the database must handle 50,000 QPS during warm-up instead of its usual 2,500. If the database can sustain this for several minutes, gradual warm-up is the simplest approach. The risk: if the database cannot handle the spike, the system degrades before the cache has time to warm.

Controlled ramp-up. Route traffic to the new cache node gradually. Start at 10% of traffic, let it warm for a few minutes, then increase to 25%, 50%, and finally 100%. During the ramp, the old cache (or database) still handles the remaining traffic. This limits the cold-start spike to a fraction of total load.

Load balancers or service meshes can implement this with weighted routing. The tradeoff is operational complexity—you need tooling to manage the ramp and monitoring to decide when to increase the weight.

Pre-loading (cache priming). Before accepting traffic, load the most popular keys into the new cache node. This requires knowing which keys are hot. Sources include:

- Access logs from the last hour (extract the most-requested keys)
- A popularity index maintained in the database (a counter table tracking access frequency)
- A snapshot from an existing replica (copy the key-value pairs directly)

Pre-loading gets the hit rate to a reasonable level (often 70–80% of steady-state) before the node serves real traffic. The remaining keys populate through natural misses. This approach is common in large-scale systems where a full cold start would overwhelm the database.

Protecting the Database During Warm-Up

Even with warm-up strategies, some period of elevated database load is unavoidable. Two mechanisms prevent the spike from cascading:

Rate limiting on the miss path. Cap the number of concurrent database queries triggered by cache misses. If the limit is 5,000 QPS and 50,000 misses arrive, 45,000 requests wait or receive a degraded response (stale data from a secondary source, or a brief error). The database stays within its capacity.

Request coalescing. When multiple requests miss on the same key simultaneously, only one query goes to the database. The others wait for the result. This is the same singleflight pattern used to prevent stampedes (covered in the failure modes article). During cold start, it's especially effective because many users request the same popular keys.

Monitoring Cold Starts

The hit rate is the primary indicator. A sudden drop from steady-state (say, 95%) toward 0% signals a cold start event. Correlate with deployment events, node replacements, or cache flush operations to identify the cause.

Track the warm-up curve: how quickly the hit rate recovers. A healthy warm-up reaches 80% of steady-state within minutes for most workloads. If recovery is slow, the working set may be too large for natural traffic to cover—pre-loading would help.

From here, specific failure patterns—cache penetration, avalanche, and stampede—show what can go wrong and how the defenses prevent them from taking down the system.